Universidad ORT Uruguay
Facultad de Ingeniería

Entregado como requisito para la obtención del título de
Ingeniería en Sistemas

Juan Pedro Ferreira (292400)
Matías Trujillo (272561)
Agustín Lanata (285326)

Entrega 1
Diseño de Aplicaciones 2

2025

Índice

Evidencia del diseño y especificación de la API **3**
Principios REST aplicados 3
1. Interfaz uniforme: 3
2. Comunicación sin estado (Stateless): 3
3. Arquitectura en capas: 3
4. Modelo Cliente-Servidor: 4
5. Cacheable: 4
La API implementa un uso correcto de los códigos de estado HTTP 4
Manejo centralizado de errores con ExceptionFilter: 4

Evidencia del diseño y especificación de la API

Nuestra aplicación ha sido diseñada y desarrollada conforme a los principios de la arquitectura REST (Representational State Transfer), lo cual garantiza una interfaz uniforme, predecible y fácilmente consumible por clientes externos.

Principios REST aplicados

1. Interfaz uniforme:

La API sigue el principio de interfaz uniforme, donde todos los recursos del sistema son accedidos y manipulados de manera coherente mediante las operaciones estándar del protocolo HTTP. Se utilizan los métodos GET para la recuperación de recursos, POST para la creación, PUT para la actualización y DELETE para la eliminación.

Se refiere a tener coherencia a lo largo de toda la interfaz, por ejemplo en nuestro caso todas las asociaciones decidimos darle un status code de 201, representamos los recursos dentro de las uri en minúscula y en plural seguido si es necesario por su id.

2. Comunicación sin estado (Stateless):

La arquitectura de la API es stateless, lo que significa que el servidor no guarda información entre solicitudes. Cada petición debe contener toda la información necesaria para ser procesada.Este enfoque permite mayor escalabilidad, ya que cualquier instancia del servidor puede atender cualquier solicitud sin depender de datos anteriores. Además, facilita un diseño desacoplado y más simple.Por eso, en el programa se utiliza la inyección de dependencias con servicios registrados como scoped. Esto asegura que en cada solicitud se cree una nueva instancia del servicio, manteniendo el principio de independencia entre peticiones.

También se agrega un filtro de excepciones global con options.Filters.Add(typeof(ExceptionFilter)). Esto permite manejar errores de forma consistente, sin romper la lógica stateless de la aplicación.

3. Arquitectura en capas:

La aplicación está diseñada siguiendo una arquitectura en capas, lo que promueve la separación de responsabilidades y mejora la organización del código. Esta estructura incluye una capa de presentación compuesta por los controladores, una capa de lógica de negocio que encapsula las reglas del sistema, y una capa de acceso a datos encargada de interactuar con la base de datos. Esta separación permite desarrollar, probar y mantener cada parte del sistema de manera independiente, lo que incrementa la escalabilidad y facilita futuras modificaciones o extensiones en la aplicación.

4. Modelo Cliente-Servidor:

La arquitectura establece una separación clara entre el cliente y el servidor. El cliente realiza solicitudes (por ejemplo, GET /class/{id}) y el servidor responde con los recursos o el resultado de las operaciones. Esta división promueve una evolución independiente de ambos componentes y mejora la portabilidad del cliente.

5. Cacheable:

En el futuro, se podría exponer un endpoint GET /primitive-types que sea cacheable, ya que estos datos cambian muy poco. Esto mejoraría el rendimiento al evitar llamadas repetidas por información estática.

Versionado

Tener un sistema de versionado permite llevar un control ordenado de los cambios realizados, facilita la identificación y corrección de errores, y posibilita volver a versiones anteriores si es necesario. Además, ayuda a mantener la compatibilidad con los usuarios al saber qué versión están utilizando, mejora la gestión del despliegue y mantenimiento en distintos entornos, y permite documentar y comunicar de manera clara las mejoras o modificaciones realizadas.

La API implementa un uso correcto de los códigos de estado HTTP

Asegura que cada respuesta refleja con precisión el resultado de la operación solicitada. Esto permite a los clientes interpretar adecuadamente las respuestas del servidor y actuar en consecuencia, ya sea ante un éxito, un error de entrada o un fallo inesperado.

Para operaciones exitosas, se emplean códigos del grupo 2xx. Por ejemplo, el código 200 OK indica que la solicitud se procesó correctamente, como al obtener información de una clase o actualizar un atributo. El código 201 Created se utiliza cuando un recurso ha sido creado exitosamente, como al registrar un nuevo método, en nuestro caso lo utilizamos también cuando por ejemplo en un método se asocia otro método lo mismo cuando se asocia un atributo a una clase. Por otro lado, 204 No Content se usa cuando una operación de eliminación se completa con éxito, sin necesidad de enviar contenido adicional en la respuesta, como al eliminar un atributo.

En el caso de errores generados por el cliente, se utilizan códigos 4xx junto con un conjunto de excepciones personalizadas que permiten generar respuestas claras y específicas. El código 400 Bad Request se devuelve cuando los datos son inválidos o malformados, como puede suceder ante excepciones como ClassException o InvalidAttributeException. Si el recurso solicitado no existe, se retorna un 404 Not Found, manejando casos como ClassNotFoundException o ParameterNotFoundException. Asimismo, 409 Conflict se utiliza para indicar conflictos lógicos en la operación, por ejemplo, si se intenta agregar un atributo ya existente en una clase o eliminar una clase que está en uso.

Finalmente, los errores inesperados del servidor se representan con el código 500 Internal Server Error, garantizando que, aún sin un mapeo específico, el cliente reciba una respuesta coherente y consistente frente a fallos internos no controlados. Esta estrategia mejora la robustez de la API y facilita el desarrollo del cliente al ofrecer respuestas predecibles y adecuadas a cada situación.

Manejo centralizado de errores con ExceptionFilter:

La aplicación implementa un filtro global de excepciones que intercepta automáticamente los errores no controlados durante las solicitudes. Este filtro identifica el tipo de excepción, la mapea al código HTTP correspondiente y genera respuestas uniformes. Esto mejora la experiencia del usuario, facilita el consumo de la API y evita duplicar la lógica de manejo de errores en los controladores, promoviendo un diseño más limpio y desacoplado.

Link: Especificacion de Api

Vínculo : https://docs.google.com/spreadsheets/d/11NG2_2hbvRpVz3SdVf3stvxhpIQ5ezOxXPpVIGzhP7w/edit?usp=sharing